

Introduction

A practical machine learning project built alongside learning ML algorithms in theory.

Goal: Predict match outcomes (**H = Home Win**, **D = Draw**, **A = Away Win**) using historical football match data and progressively improve the dataset as new ML algorithms are learned.

Roadmap

Part	Focus	Key Outcome
1	Foundations & first XGBoost attempt	Discovered the model was ignoring Draws
2	Improving XGBoost	Chronological validation, tuning, thresholding → Draw F1 0.29
3	Expanding the dataset	Found an information ceiling — more data ≠ more accuracy
4	The mathematics of Dixon-Coles	Implemented a Poisson-based goal model from scratch
5	The hybrid predictor	Blended XGBoost + Dixon-Coles → Draw F1 0.36

TL;DR — The goal was never chasing a leaderboard score. It was to use a real Premier League dataset as a playground for genuinely understanding gradient boosting, probabilistic modelling, and rigorous evaluation — not just importing `sklearn` and hoping for the best.

Part 1 — Foundations & the First XGBoost Attempt

Project Goal

The goal of this project was never simply to obtain the highest possible prediction accuracy.

Instead, the objective was to use a **real football dataset** as a playground while learning Machine Learning algorithms in depth.

Rather than learning XGBoost, Gradient Boosting and probabilistic models only from theory, every concept was implemented on an actual Premier League dataset and improved step-by-step through experimentation.

This section documents everything that happened after the completion of the ensemble-learning section (Decision Trees, Random Forests and AdaBoost).

Starting Point

At the beginning of this phase the dataset consisted of only the **2025/26 Premier League season**.

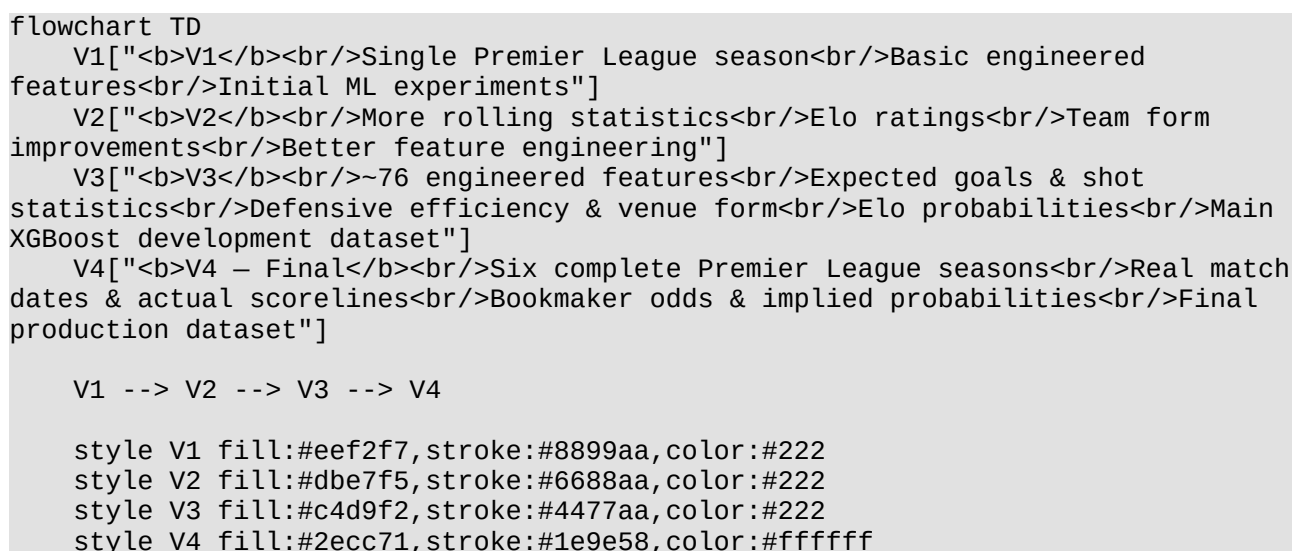
Over time the project evolved into a complete historical dataset containing **six Premier League seasons (2020/21 → 2025/26)** with more than seventy engineered features.

This was not planned from the beginning — it happened naturally as limitations of the model became clear.

Evolution of the Dataset

One of the biggest lessons of this project was that **better models cannot compensate for poor data**.

The dataset itself went through multiple revisions:



The final dataset was renamed to

Football_features_2020-2026 (V4).csv

and became the final version used throughout the remainder of the project.

Moving from Gradient Boosting to XGBoost

The next algorithm studied was **Extreme Gradient Boosting (XGBoost)**.

Unlike Random Forests, which reduce variance by averaging many independent trees, XGBoost builds trees **sequentially**, with every new tree attempting to correct the mistakes of the previous ensemble.

Mathematically:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

where

- F_{m-1} = previous ensemble
- h_m = newly learned tree
- η = learning rate

Instead of trying to build one perfect tree, XGBoost repeatedly learns the **residual errors** remaining after previous trees.

This produces a much stronger learner while still controlling overfitting through regularisation.

First XGBoost Results

The initial XGBoost implementation immediately outperformed the previous tree-based models.

However, one unexpected issue became obvious.

Although overall accuracy looked reasonable, the model had effectively learned to ignore **Draws**.

Since home wins occur most frequently in football, the model simply learned that predicting Home or Away was usually "safe," while predicting Draw was too risky.

The result was a model that achieved acceptable accuracy while producing almost **zero Draw predictions**.

△ This became the first major bottleneck of the project.

□ First Round of Improvements

Several improvements were explored before making major architectural changes. These included:

- Hyperparameter tuning
- `TimeSeriesSplit` validation
- Chronological train/test splitting
- Class balancing
- Different objective functions
- Manual probability thresholding
- Feature engineering targeted specifically at Draw prediction

Rather than blindly accepting higher accuracy, every experiment was analysed using:

- Accuracy
- Precision
- Recall
- F1-score

- Confusion Matrix
- Macro F1

This shifted the project away from chasing one metric and toward evaluating the entire classifier.

Part 2 — Improving XGBoost

The First Problem

The first implementation of XGBoost was encouraging.

Compared to the previous tree-based algorithms, XGBoost achieved the highest overall accuracy while requiring relatively little feature preprocessing.

However, a deeper inspection of the confusion matrix exposed a serious weakness.

The classifier had effectively learned to **avoid predicting Draws altogether**.

Because Home wins occur most frequently in the Premier League, the model naturally learned that predicting **Home** or **Away** usually resulted in a lower loss than risking a Draw prediction.

The consequence was a deceptively good accuracy while producing an almost useless Draw classifier.

This marked the beginning of the optimisation phase.

Making the Evaluation More Realistic

The original implementation evaluated the model using a normal train/test split.

Although `shuffle=False` preserved chronology, hyperparameter tuning still required improvement.

Instead of using ordinary cross-validation, the project switched to `TimeSeriesSplit`, which performs validation chronologically — each fold trains on an expanding window of the past and validates on the matches that follow it:

```
flowchart LR
    subgraph F1["Fold 1"]
        direction LR
        T1[Train] --> Val1[Validate]
    end
    subgraph F2["Fold 2"]
        direction LR
        T2["Train (larger)"] --> Val2[Validate]
    end
    subgraph F3["Fold 3"]
        direction LR
        T3["Train (largest)"] --> Val3[Validate]
    end
    F1 --> F2 --> F3
```

This better simulated the real prediction task: **future matches should never influence predictions on earlier matches.**

Using chronological validation produced a slightly lower accuracy than before, but the results became significantly more trustworthy.

This became the permanent evaluation strategy for the remainder of the project.

Hyperparameter Optimisation

Rather than relying on XGBoost's default parameters, an extensive `GridSearchCV` optimisation was performed.

The following parameters were explored:

Hyperparameter	Purpose
<code>max_depth</code>	Controls tree complexity
<code>learning_rate</code>	Shrinks contribution of each tree
<code>n_estimators</code>	Number of boosting rounds
<code>min_child_weight</code>	Controls minimum leaf size
<code>gamma</code>	Minimum loss reduction required before splitting
<code>reg_alpha</code>	L1 regularisation
<code>reg_lambda</code>	L2 regularisation

After experimentation the best-performing configuration became:

```
objective = "multi:softprob"

gamma = 0
learning_rate = 0.01
max_depth = 2
min_child_weight = 3
n_estimators = 300
reg_alpha = 0.5
reg_lambda = 3
```

Interestingly, the optimal model preferred:

- shallow trees
- heavy regularisation
- many boosting rounds
- a very small learning rate

which suggested that slow, conservative learning generalised better than deep, highly expressive trees.

Switching from `multi:softmax` to `multi:softprob`

Originally the model used:

```
objective = "multi:softmax"
```

which immediately outputs a class label.

This was later replaced with:

```
objective = "multi:softprob"
```

Instead of simply returning:

```
Home
```

the model now produced a full probability distribution:

```
Away : 0.24  
Draw : 0.31  
Home : 0.45
```

This change was extremely important. Instead of accepting the class with the highest probability, the project could now manipulate probabilities directly.

Without probability outputs, none of the later threshold tuning or blending work would have been possible.

Handling Class Imbalance

Another observation was that Draws made up only approximately:

Outcome	Share of Dataset
Home	43%
Away	33%
Draw	24%

To reduce this imbalance:

```
compute_sample_weight(class_weight="balanced")
```

was introduced during training. Each Draw sample now contributed a larger loss during optimisation. The intention was simple:

Make mistakes on Draw matches more expensive than mistakes on Home wins.

This immediately increased Draw recall, although accuracy decreased slightly. The trade-off was considered worthwhile.

Manual Draw Thresholding

After switching to `multi:softprob`, probability thresholding became possible.

Instead of:

```
prediction = np.argmax(probabilities)
```

the project introduced:

```
if draw_probability >= threshold:
    predict Draw
else:
    predict argmax
```

Several thresholds were tested:

Draw Threshold	Observation
0.30	Highest Draw recall, but too many Away wins became Draws
0.35	Best overall balance
0.40	Similar to original argmax behaviour
0.45	Draw predictions became too conservative

One particularly interesting discovery was that lowering the threshold caused many predicted Away wins to become Draws — visible directly in the confusion matrices.

Rather than increasing overall accuracy, threshold tuning allowed the project to deliberately **trade accuracy for a much stronger Draw classifier**.

Results After Threshold Optimisation

The best threshold (≈ 0.35) produced approximately:

Metric	Value
Accuracy	43.9%
Macro F1	≈ 0.42
Draw F1	0.29

Compared to the original XGBoost implementation, overall accuracy dropped slightly, but Draw prediction improved dramatically — a far more balanced classifier.

Additional Feature Engineering Attempts

At this stage it seemed likely that the bottleneck was insufficient information.

Several new handcrafted features were introduced, including ideas specifically designed to capture

Draw behaviour, such as:

- Draw propensity
- Team closeness metrics
- Head-to-head draw statistics
- Difference-based features

Surprisingly... **none of these produced any measurable improvement.**

This was an important lesson: adding features is not automatically useful. If the existing feature set already captures the available information, additional engineered variables often become redundant. These experiments were therefore abandoned.

Key Lesson

This phase completely changed the direction of the project.

Initially the assumption was:

Better hyperparameters will solve the problem.

After weeks of experimentation, the conclusion became:

The optimisation procedure was no longer the bottleneck.

The model itself was already learning effectively. The real limitation appeared to be the **information available before kickoff.**

This motivated the next major stage of the project: **improving the dataset itself rather than continuing to tune XGBoost.**

Part 3 — Expanding the Dataset & Discovering the Information Ceiling

A Shift in Thinking

By this point it had become clear that XGBoost itself was **not the limiting factor.**

Multiple rounds of hyperparameter tuning, probability thresholding and feature engineering had produced only incremental improvements. The obvious next question became:

"What if the model simply doesn't have enough information?"

Instead of continuing to optimise the algorithm, the focus shifted toward **improving the dataset itself.** This ended up becoming one of the biggest turning points of the entire project.

Expanding Beyond One Season

Originally the project only contained matches from the **2025/26 Premier League season**.

Although the engineered features were reasonably good, a single season provides only:

- 380 matches
- limited tactical variation
- limited team history
- relatively little data for a boosting algorithm

To improve generalisation, the dataset was expanded to include every Premier League season from **2020/21** through **2025/26**, bringing the dataset to:

2,280 Premier League matches across six complete seasons.

This immediately provided a much richer training set while still keeping the league consistent.

Collecting Historical Data

The original feature dataset unfortunately did not contain several pieces of information that would later become essential, including:

- Match dates
- Actual scorelines
- Bookmaker odds
- Implied bookmaker probabilities

Rather than collecting these manually, AI was used to automate the process.

Historical Premier League data was downloaded from **football-data.co.uk**, after which AI automatically:

- matched every fixture
- validated team names
- checked for duplicate matches
- verified one-to-one joins
- merged every season together
- produced a clean dataset

The final merged dataset became:

`Football_features_2020-2026 (V4).csv`

This saved many hours of repetitive work while ensuring every merge could be verified.

New Information Added

The final dataset now contained considerably more information than any previous version. Additional columns included:

```
Date
Home_goals_actual
Away_goals_actual

Odds_Home
Odds_Draw
Odds_Away

Odds_implied_prob_home
Odds_implied_prob_draw
Odds_implied_prob_away
```

These additions would later make probabilistic football models such as Dixon-Coles possible.

Rebuilding XGBoost

With the expanded dataset complete, the entire XGBoost pipeline was rebuilt, including:

- chronological train/test split
- `TimeSeriesSplit` validation
- balanced sample weights
- tuned hyperparameters
- probability thresholding

The expectation was that significantly more historical information would finally produce a major jump in predictive performance.

Surprisingly... it didn't.

Testing Bookmaker Odds

One of the biggest experiments involved introducing bookmaker odds directly into the feature set.

Intuitively, bookmaker odds contain enormous amounts of information — generated using statistical models, injury news, betting markets, expert adjustments, and public sentiment.

The expectation was therefore:

Adding bookmaker odds should dramatically improve prediction accuracy.

Instead, the improvement was almost nonexistent:

Model	Accuracy	Draw F1
XGBoost (without odds)	43.9%	0.29

Model	Accuracy	Draw F1
XGBoost (with odds)	43.6%	0.30

The overall prediction quality barely changed.

An Unexpected Discovery

Although performance hardly improved, feature importance told a completely different story.

Bookmaker odds immediately became some of the **highest-ranked features** inside XGBoost.

At first this appeared contradictory. However, the explanation became surprisingly satisfying: the model already possessed Elo ratings, rolling form, expected goals, shooting statistics, defensive efficiency, and positional information — features that already encoded much of the same information bookmakers use when pricing matches.

Therefore, the odds were **important**, but they were **mostly redundant**.

Rather than exposing poor feature engineering, this experiment actually validated that the existing feature engineering was already capturing much of football's underlying structure — one of the strongest confirmations that the earlier engineering work had been worthwhile.

Chasing the Information Ceiling

Over several hours of experimentation, a clear pattern emerged. Every reasonable improvement followed roughly the same trend:

```
flowchart LR
    A[Better Hyperparameters] -->|small improvement| B[Better Features]
    B -->|small improvement| C[Six Seasons of Data]
    C -->|almost no improvement| D[Bookmaker Odds]
    D --> E["Information Ceiling"]

    style E fill:#e74c3c,stroke:#b23324,color:#ffffff
```

Every path seemed to converge toward roughly the same performance level. This suggested something much more interesting than a poorly tuned model — it suggested the existence of an **information ceiling**.

Given only pre-match aggregate statistics, there may simply not be enough information available to predict football matches substantially better.

Looking Beyond Machine Learning

At this point, the project reached an important crossroads.

Traditional machine learning had been pushed surprisingly far. The remaining question became:

"Should football itself be modelled differently?"

Rather than learning purely from engineered features, perhaps the model should learn directly from **goal scoring distributions**.

This led to the discovery of the **Dixon-Coles model**, one of the most influential probabilistic models in football analytics.

Unlike XGBoost, which learns patterns from features, Dixon-Coles attempts to model the actual process that generates football scores. This marked the beginning of the final phase of the project.

Part 4 — Understanding the Dixon-Coles Model (*The Mathematics Behind It*)

"Machine Learning gave us a powerful black-box model. Dixon-Coles taught us the mathematics of football."

Why Learn Another Model?

Up until this point the project had been entirely based around **Machine Learning**.

The XGBoost model looked at dozens of engineered features and learned complicated relationships between them. Although extremely powerful, it has one disadvantage:

It never explicitly models **how goals are scored**.

Football is fundamentally a goals game. Instead of asking *"Who is likely to win?"*, the Dixon-Coles model asks:

"How many goals will each team score?"

Once we know that probability distribution, obtaining Home Win / Draw / Away Win probabilities becomes simple mathematics.

Why Dixon-Coles?

The Dixon-Coles model is one of the most influential statistical football models ever created. Unlike XGBoost, it:

- is interpretable
- is mathematically elegant
- explains *why* predictions happen
- produces calibrated probabilities naturally

Most importantly, it models football using probability theory instead of decision trees.

Before Implementation

Before writing a single line of code, I intentionally stopped.

Instead of copying an implementation from GitHub, I wanted to actually understand the mathematics. This required learning several concepts that I had never studied before.

Topics Learned

- Poisson Distribution
- Probability Mass Functions
- Numerical Optimization
- Attack & Defensive Rating Models
- Dixon-Coles Low Score Correction
- L-BFGS Optimization

Overall I spent roughly:

- **2 hours** understanding the statistical theory
- **3 hours** implementing the entire algorithm from scratch

No libraries exist that simply expose a `fit_dixon_coles()` function, the way Scikit-Learn exposes Random Forest or Logistic Regression. The model had to be implemented manually.

Step 1 — The Poisson Distribution

The core assumption is simple: football goals are relatively rare events. Rare events are often modelled extremely well using the Poisson Distribution.

Instead of saying "*Liverpool will score exactly 2 goals*," the model predicts an entire distribution:

Goals	Probability
0	0.08
1	0.24
2	0.31
3	0.22
4	0.10
5+	Remaining

Notice something important — the model predicts **an entire probability distribution** rather than one number.

The Poisson Formula

For a team expected to score λ goals, the probability of scoring exactly k goals is:

$$P(X=k) = \frac{e^{-\lambda} \lambda^k}{k!}$$

where

- λ = expected goals
- k = actual goals scored

Example

Suppose Liverpool have $\lambda = 2.1$. The model automatically computes $P(0 \text{ goals})$, $P(1 \text{ goal})$, $P(2 \text{ goals})$, and so on, using the Poisson equation — exactly what `poisson.pmf(...)` does.

Step 2 — Team Strength Parameters

The next question becomes: *where does λ come from?*

Instead of using xG directly, Dixon-Coles estimates hidden team strengths. Every club has:

- **Attack Strength** (α)
- **Defensive Strength** (β)

Additionally, the league has:

- **Home Advantage** (γ)

The expected home goals become:

$$\lambda = e^{(\alpha_{\text{home}} + \beta_{\text{away}} + \gamma)}$$

The expected away goals become:

$$\mu = e^{(\alpha_{\text{away}} + \beta_{\text{home}})}$$

Notice that nothing is hand-written — the optimizer learns every parameter automatically.

Step 3 — Why Exponentials?

Suppose the optimizer predicts:

Attack	=	-0.8
Defense	=	0.4
Home Advantage	=	0.2

Adding these gives -0.2. Goals cannot be negative — exponentials solve this instantly:

$$e^{-0.2} = 0.818$$

No matter what the optimizer learns, expected goals always stay positive.

Step 4 — Maximum Likelihood Estimation

Now we need to determine every team's attack, every team's defence, home advantage, and (later) ρ .

Instead of guessing them, the optimizer asks:

Which values make the historical matches **most likely**?

For every historical match, it computes:

$$P(\text{Home Goals}) \times P(\text{Away Goals})$$

using Poisson distributions. Those probabilities are multiplied together across every match, and the optimizer searches for the parameter values that maximise this overall likelihood.

Why Negative Log Likelihood?

Multiplying thousands of tiny probabilities produces numbers so small that computers underflow. Instead, we use logarithms — multiplication becomes addition.

Instead of *maximizing* Likelihood, we *minimize* Negative Log Likelihood, which is mathematically equivalent:

$$-\sum \log(P(\text{match}))$$

This is far more numerically stable.

Step 5 — Numerical Optimization

The Dixon-Coles model has dozens of unknown parameters. For a 20-team league, roughly:

- 20 attack values
- 20 defence values
- 1 home advantage parameter
- 1 correlation parameter (ρ)

The optimizer therefore searches over **40+ dimensions simultaneously** — impossible manually. Instead, the project uses:

```
scipy.optimize.minimize(method="L-BFGS-B")
```

Why L-BFGS-B?

Unlike ordinary Gradient Descent, L-BFGS-B uses gradients *and* an approximation of the Hessian matrix. This allows it to estimate not only "*which direction is downhill?*" but also "*how curved is the surface?*"

As a result, it usually converges much faster than basic gradient descent while using far less memory than a full Newton optimizer.

Step 6 — The Dixon-Coles Correction

A plain Poisson model assumes Home Goals and Away Goals are independent. Real football isn't perfectly independent — low-scoring games behave slightly differently, especially:

- 0–0
- 1–0
- 0–1
- 1–1

Dixon & Coles introduced a correction factor $\tau(x, y, \lambda, \mu, \rho)$ which only modifies these four scorelines. Every other score remains unchanged.

The correction is controlled by ρ :

- If $\rho = 0$, there is **no correction**.
- **Negative** values increase draw probability.
- **Positive** values decrease it.

An Interesting Discovery

One of the most surprising findings during implementation was that the fitted model converged to:

$\rho \approx 0.0019$

Initially this looked suspicious. Additional simulations showed that the fitted Poisson model already predicted low-score matches almost perfectly — there was almost nothing left for Dixon-Coles to correct.

This is actually consistent with several modern studies suggesting that the original 1990s Dixon-Coles correction has become much weaker in modern Premier League football.

Rather than indicating a bug, it showed that the plain double-Poisson model already fit this dataset extremely well.

□ Mathematical Pipeline

```

flowchart TD
    A[Historical Matches] --> B["Estimate  $\alpha$ ,  $\beta$ ,  $\gamma$ "]
    B --> C["Compute  $\lambda$  and  $\mu$ "]
    C --> D["Poisson Goal Probabilities"]
    D --> E["Apply  $\tau$  Correction"]
    E --> F[Likelihood]
    F --> G["Negative Log Likelihood"]
    G --> H["L-BFGS Optimizer"]
    H --> I[Final Team Parameters]

    style I fill:#2ecc71,stroke:#1e9e58,color:#ffffff

```

What This Section Added

This stage transformed the project from simply using machine learning into understanding one of football analytics' most influential statistical models.

Instead of treating football prediction purely as a classification problem, the project now models the underlying goal-scoring process itself using probability theory, maximum likelihood estimation, and numerical optimization. Implementing the algorithm from scratch provided a much deeper understanding of statistical modelling than simply importing another machine learning library.

Part 5 — Implementing the Hybrid Predictor

"XGBoost learned patterns from engineered features. Dixon-Coles modelled how football is actually played. The final step was combining both into one hybrid model."

Why Blend Two Models?

After completing the Dixon-Coles implementation, the project now had two fundamentally different prediction systems.

Model 1 — XGBoost

Uses:

- Elo ratings
- Team form
- Expected goals
- Shots
- Corners
- League position
- ~75 engineered features

Strengths:

- Learns complex nonlinear relationships
- Excellent overall classification model
- Makes use of rich feature engineering

Weaknesses:

- Doesn't explicitly model goals
- Originally struggled badly with predicting draws

Model 2 — Dixon-Coles

Uses:

- Historical scorelines only

Learns:

- Team attack strength
- Team defensive strength

- Home advantage
- Goal distributions

Strengths:

- Statistically interpretable
- Produces naturally calibrated probabilities
- Models football at the goal level

Weaknesses:

- Ignores all engineered features
- Doesn't know about current form, Elo, injuries, shots, etc.

These two models make mistakes for completely different reasons — that made them perfect candidates for an ensemble.

The Idea Behind Blending

Instead of allowing one model to make the prediction, each model contributes a percentage of confidence:

$$P_{\text{blend}} = w_1 P_{\text{XGB}} + w_2 P_{\text{DC}}$$

where

$$w_1 + w_2 = 1$$

Worked example — suppose XGBoost predicts:

Outcome	Probability
Away	0.20
Draw	0.30
Home	0.50

while Dixon-Coles predicts:

Outcome	Probability
Away	0.15
Draw	0.45
Home	0.40

Using a **30% XGBoost / 70% Dixon-Coles** split, the blended probabilities become:

Outcome	Probability
Away	0.165
Draw	0.405
Home	0.430

The prediction is then taken from these new probabilities.

Why 30% / 70%?

This ratio was **not chosen arbitrarily**.

During experimentation, Claude was used as a research assistant to run numerous simulations and compare different blending ratios without requiring the entire model to be retrained each time.

Multiple combinations were evaluated before arriving at **30% XGBoost / 70% Dixon-Coles** as the most promising compromise between:

- overall accuracy
- macro F1
- draw prediction

This dramatically reduced experimentation time and allowed many candidate blends to be tested efficiently.

Implementing the Blend

The implementation itself is surprisingly small:

```
blend_probs = (  
    0.30 * xgb_test_probs  
    +  
    0.70 * dc_test_probs  
)
```

Both probability matrices have shape `(number_of_matches, 3)`, with each row containing `[Away, Draw, Home]`.

Because both models use the exact same ordering, NumPy performs the weighted average element-wise.

The Draw Threshold

Earlier in the project, one major issue became obvious: using `np.argmax()` almost always ignored draws.

Even when:

```
Draw = 34%  
Home = 35%  
Away = 31%
```

the prediction became **Home** — despite the match being highly draw-like.

The solution was to introduce a manual draw threshold:

```
DRAW_THRESHOLD = 0.27
```

Prediction logic became:

```
if draw_probability >= DRAW_THRESHOLD:
    predict Draw
else:
    predict highest_probability
```

This simple rule dramatically improved draw recall while sacrificing only a small amount of overall accuracy.

Final Prediction Pipeline

```
flowchart TD
    A["Historical Dataset"] --> B[XGBoost]
    A --> C[Dixon-Coles]
    B --> D[Probabilities]
    C --> E[Probabilities]
    D --> F["Weighted Probability Blend<br/>30% XGBoost + 70% Dixon-Coles"]
    E --> F
    F --> G["Draw Threshold ≥ 0.27"]
    G --> H[Final Match Prediction]

    style H fill:#2ecc71,stroke:#1e9e58,color:#ffffff
    style F fill:#fdf2d0,stroke:#d8ae2c,color:#222
```

Final Results

XGBoost (Best Standalone)

Metric	Value
Accuracy	43.9%
Macro F1	≈0.42
Draw F1	0.29

Dixon-Coles (Standalone)

Metric	Value
Accuracy	≈47.6%
Macro F1	≈0.36
Draw F1	≈0.16

Final Hybrid Model

Metric	Value
Accuracy	44.1%

Macro F1 **0.43**

Draw F1 **0.36**

Dixon-Coles alone had surprisingly strong overall accuracy but still struggled to identify draws consistently.

Using **30% XGBoost / 70% Dixon-Coles** with a **Draw Threshold of 0.27**, the hybrid model sacrificed a small amount of raw accuracy compared to standalone Dixon-Coles, but achieved the **best overall balance across all three classes** and substantially improved draw prediction.

Understanding the Result

An important conclusion emerged from this project. Three very different modelling approaches were tested:

- Gradient Boosted Trees (XGBoost)
- Statistical Goal Model (Dixon-Coles)
- Hybrid Ensemble

Despite being fundamentally different algorithms, **all converged to roughly the same performance level** on the available pre-match information.

This strongly suggests that the project is approaching the practical **information ceiling** of this dataset rather than being limited by model choice alone.

What This Project Achieved

This project began as an exercise to apply machine learning algorithms learned in theory to a real-world dataset.

Over the course of roughly a month, it evolved into a complete football analytics pipeline involving:

- extensive feature engineering
- chronological validation
- probability calibration through thresholding
- dataset expansion across six Premier League seasons
- bookmaker odds integration
- implementation of the Dixon-Coles statistical model from scratch
- hybrid ensemble modelling

Along the way, AI tools played an important supporting role. Claude assisted by running large numbers of simulation experiments, comparing alternative modelling strategies, and helping identify promising directions for further testing. AI also automated the otherwise tedious task of merging six seasons of historical match data, bookmaker odds, scorelines and dates into the final dataset (**Football_features_2020-2026 (V4)**), allowing more time to focus on understanding the underlying mathematics and machine learning concepts rather than repetitive data

manipulation.

The final result is more than just a predictive model — it represents a complete end-to-end machine learning project, demonstrating data collection, feature engineering, statistical modelling, ensemble learning, validation, experimentation and critical analysis on a real-world sports analytics problem.